COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/17/26	**DATE SUBMITTED:**	9/17/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 Implimenting trees

1. PROCEDURE OUTPUT

main.cpp | binaryTree.h

hoa9.1 > main.cpp > main()

```cpp
#include <iostream>
#include "binaryTree.h"

int main() {

    BinaryTree<char>* root = new BinaryTree<char>('A');
    root->insertChildLeft('B');
    root->insertChildRight('C');

    root->getLeftChild()->insertChildLeft('D');
    root->getLeftChild()->insertChildRight('E');
    root->getRightChild()->insertChildRight('D');

    // Display Root and Immediate Children
    std::cout << "The root: " << root->getRootVal() << std::endl;
    std::cout << "The left child of root a is: " << root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The right child of root a is: " << root->getRightChild()->getRootVal() << std::endl;

    // Display Sub-children (Children of 'B')
    std::cout << "The left child of B is: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The right child of B is: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    // Update 'C' to 'V' and print result
    root->getRightChild()->setRootVal('V');
    std::cout << "Updated right child of root a is: " << root->getRightChild()->getRootVal() << std::endl;
    std::cout << "The right child of V is: " << root->getRightChild()->getRightChild()->getRootVal() << std::endl;

    return 0;
}
```

main.cpp | binaryTree.h

hoa9.1 > binaryTree.h > ...

```cpp
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree {
private:
    T key;
    // pointers for left and right child
    BinaryTree<T>* leftChild;
    BinaryTree<T>* rightChild;

public:
    // constructor:
    BinaryTree(T rootObj);

    void insertChildLeft(T newNode);
    void insertChildRight(T newNode);
    BinaryTree<T>* getLeftChild();
    BinaryTree<T>* getRightChild();
    T getRootVal();
    void setRootVal(T val);
};

// constructor:
template <typename T>
BinaryTree<T>::BinaryTree(T rootObj) {
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}

// insert child:
template <typename T>
void BinaryTree<T>::insertChildLeft(T newNode) {
    if (leftChild == nullptr) {
        leftChild = new BinaryTree<T>(newNode);
    } else {
        BinaryTree<T>* tempNode = new BinaryTree<T>(newNode);
        // move the current left child below the new Node;
        tempNode->leftChild = leftChild;

        leftChild = tempNode;
    }
}

template <typename T>
void BinaryTree<T>::insertChildRight(T newNode) {
    if (rightChild == nullptr) {
        rightChild = new BinaryTree<T>(newNode);
    } else {
        BinaryTree<T>* tempNode = new BinaryTree<T>(newNode);
        tempNode->rightChild = rightChild;

        rightChild = tempNode;
    }
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getLeftChild() {
    return leftChild;
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getRightChild() {
    return rightChild;
}

template <typename T>
T BinaryTree<T>::getRootVal() {
    return key;
}

template <typename T>
void BinaryTree<T>::setRootVal(T val) {
    key = val;
}

#endif
```

```
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes> cd "c:\Users\J
f ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
The root: A
The left child of root a is: B
The right child of root a is: C
The left child of B is: D
The right child of B is: E
Updated right child of root a is: V
The right child of V is: D
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\hoa9.1>
```

- **BinaryTree(T rootObj)** This is the constructor I implemented to initialize a new tree node. It sets the node's key attribute to rootObj and assigns both leftChild and rightChild pointers to nullptr, indicating that the newly created node does not have any children yet.
- **void insertChildLeft(T newNode)** This method to attach a new node to the left side of the current node. If leftChild is currently nullptr, it simply dynamically allocates a new node. If a left child already exists, it creates the new node, links the old left child underneath it, and sets leftChild to point to this new node.
- **void insertChildRight(T newNode)** This function handles adding a node to the right branch. Similar to my left insertion logic, if rightChild is empty, it creates a new BinaryTree node. If a right child already exists, it inserts the new node in between and shifts the old right child down as a subtree under the new node.
- **BinaryTree<T>* getLeftChild()** A getter method I wrote to return the memory address stored in leftChild. I use this to access and navigate down the left branch of the tree.
- **BinaryTree<T>* getRightChild()** A getter method that returns the rightChild pointer. This allows me to traverse or access nodes on the right branch.
- **T getRootVal()** I implemented this getter to retrieve and return the actual value stored in the node's key member variable.
- **void setRootVal(T val)** This is a setter method I added so I can update or overwrite the value stored in key whenever needed (such as updating 'C' to 'V').

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, debugging support, and documentation formatting.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **AI was used strictly as a learning assistant to help structure the C++ class header and implementation files, fix `main.cpp` output execution errors, and format the class function documentation. I manually verified, compiled, and tested all code logic and wrote the activity solutions myself, ensuring the submission reflects my own technical understanding without replacing original effort or problem-solving.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.